DESIGN AND IMPLEMENTATION OF A MINI COMPILER USING PYTHON AND PLY

Submitted By

Name: Mostafa Jamal Shakib
Student ID: 202324405018

Department of Computer Science and Technology

Abstract

Compiler design is one of the most fundamental and important areas of Computer Science. A compiler acts as a translator that converts source code written in a high-level programming language into a lower-level representation that can be understood and executed by a computer system. Every modern programming language such as C, C++, Java, Python, and Rust relies on compiler technology to transform human-readable code into machine-executable instructions.

The primary objective of this project is to design and implement a Mini Compiler using Python and PLY (Python Lex-Yacc). The compiler simulates the major phases of a real compiler including Lexical Analysis, Syntax Analysis, Abstract Syntax Tree Generation, Symbol Table Construction, Intermediate Code Generation, and Code Optimization. A graphical user interface was also developed using Tkinter to visualize each phase of compilation.

Unlike traditional educational compiler projects, this implementation includes support for special-character-based identifiers such as $, #, @, &, and *. This feature demonstrates the flexibility of lexical analysis and custom language design.

The completed compiler successfully analyzes source code, generates tokens, constructs parse trees, creates symbol tables, produces intermediate code, performs optimization, and displays all results through a professional graphical interface.

Chapter 1: Introduction

A compiler is a specialized software system that translates source code written by programmers into another form that can be executed by computer hardware. Without compilers, modern software development would not be possible.

The process of compilation is divided into multiple phases. Each phase performs a specific responsibility and contributes to transforming the source code into an executable representation.

The traditional compiler phases are:

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Intermediate Code Generation
5. Code Optimization
6. Target Code Generation

In this project, the first five phases have been implemented in a simplified manner to demonstrate the core concepts of compiler construction.

The Mini Compiler developed in this project allows users to declare variables, perform arithmetic operations, generate intermediate code, and visualize internal compiler structures through an easy-to-use graphical interface.

Chapter 2: Objectives of the Project

The main objectives of this project are:

Primary Objectives

• Design a Mini Compiler using Python

• Implement Lexical Analysis using PLY

• Implement Syntax Analysis using Context Free Grammar

• Generate Abstract Syntax Trees

• Build a Symbol Table

• Generate Three Address Intermediate Code

• Implement Basic Optimization Techniques

• Develop a GUI for Compiler Visualization

Educational Objectives

• Understand Compiler Architecture

• Learn Lexical Analysis Concepts

• Learn Parsing Techniques

• Understand AST Construction

• Understand Symbol Table Management

• Learn Intermediate Code Generation

• Learn Code Optimization Concepts

Chapter 3: Problem Statement

Traditional compiler projects often focus only on token generation and syntax analysis. However, a complete compiler should demonstrate multiple compilation phases.

The challenge of this project was to create a simplified but complete compiler capable of:

• Accepting source code input

• Supporting variable declarations

• Supporting arithmetic expressions

• Supporting special character identifiers

• Displaying every compilation phase separately

• Producing optimized code

• Providing a user-friendly graphical interface

The compiler was designed to address all of these requirements.

Chapter 4: System Architecture

The system architecture of the Mini Compiler consists of seven major modules.

Source Code

↓

Lexer

↓

Parser

↓

Abstract Syntax Tree

↓

Symbol Table

↓

Intermediate Code Generator

↓

Optimizer

↓

GUI Output

Each module communicates with the next module through structured data representations.

The architecture follows the same logical workflow used in real-world compiler systems.

Chapter 5: Lexical Analysis

Lexical Analysis is the first phase of compilation.

The lexical analyzer scans the source code character by character and groups characters into meaningful units known as tokens.

A token consists of:

• Token Type

• Token Value

• Line Number

Examples:

Keyword:

int

Identifier:

a$

Number:

100

Operator:

+

The lexer was implemented using PLY Lex.

The following token categories are supported:

• Keywords

• Identifiers

• Numeric Constants

• Arithmetic Operators

• Assignment Operators

• Delimiters

• Parentheses

The lexer also tracks line numbers to improve debugging and error reporting.

Chapter 6: Special Character Identifier Support

One of the unique features of this compiler is support for variable names containing special characters.

Supported symbols:

• $

• #

• @

• &

• *

Examples:

int a$;

int b#;

int c@;

int d&;

int e*;

Regular Expression Used:

[a-zA-Z_][a-zA-Z0-9_@$#&]

This feature demonstrates how lexical rules can be customized to support language-specific identifier patterns.

In conventional programming languages, many of these identifiers would be illegal. However, in this custom compiler language they are accepted and processed successfully.

Chapter 7: Syntax Analysis

Syntax Analysis verifies whether the token sequence follows the grammar rules of the language.

The parser was implemented using PLY YACC.

The grammar supports:

Variable Declaration:

int variable;

Assignment:

variable = expression;

Expressions:

addition

subtraction

multiplication

division

parenthesized expressions

The parser constructs an Abstract Syntax Tree (AST) that represents the logical structure of the program.

If a syntax error occurs, the parser reports the exact location of the error.

Chapter 8: Abstract Syntax Tree (AST)

The Abstract Syntax Tree is one of the most important structures in compiler design.

An AST removes unnecessary grammar symbols and represents only the essential logical structure of the program.

AST Nodes Used:

ProgramNode

VarDeclNode

AssignNode

VarNode

ConstNode

BinOpNode

Example:

Source Code:

c@ = a$ + b#;

AST Representation:

AssignNode(c@)

↓

BinOpNode(+)

↓

VarNode(a$)

VarNode(b#)

The AST acts as the central data structure used by later phases of compilation.

Chapter 9: Symbol Table Construction

The Symbol Table stores information about identifiers.

Stored Information:

• Variable Name

• Variable Type

• Current Value

Example:

Variable | Type | Value

a$ | int | 10

b# | int | 20

c@ | int | 30

The symbol table enables efficient variable lookup and value tracking throughout compilation.

It is also used during intermediate code generation and optimization.

(Continue with Intermediate Code Generation, Optimization, GUI Design, Testing, Results, Advantages, Limitations, Future Work, and Conclusion on the next pages.)

Chapter 10: Intermediate Code Generation

Intermediate Code Generation is one of the most important phases of compiler design.

The primary objective of intermediate code is to create a machine-independent representation of the source program.

Instead of directly generating machine code, the compiler first generates an intermediate representation. This representation can later be optimized and translated into target code.

In this project, Three Address Code (TAC) was used as the intermediate representation.

Why Intermediate Code is Important

Intermediate code provides several advantages:

• Simplifies optimization

• Makes compiler design modular

• Supports multiple target machines

• Improves code analysis

• Reduces compiler complexity

Three Address Code

Three Address Code uses a simple instruction format:

Result = Operand1 Operator Operand2

Example:

Source Code:

c@ = a$ + b#;

Generated Intermediate Code:

t1 = a$ + b#

c@ = t1

Complex Example:

d& = (a$ + b#) * 2;

Generated TAC:

t1 = a$ + b#

t2 = t1 * 2

d& = t2

Temporary variables such as t1 and t2 are automatically generated by the compiler.

These temporary variables simplify expression evaluation and make optimization easier.

Chapter 11: Code Optimization

Optimization improves program efficiency without changing program behavior.

The objective of optimization is to reduce unnecessary computations and improve execution performance.

In this project, two optimization techniques were implemented.

Constant Folding

Constant Folding evaluates constant expressions during compilation rather than during execution.

Example:

Before Optimization:

t1 = 10 + 20

After Optimization:

t1 = 30

This reduces runtime calculations.

Constant Propagation

Constant values are propagated through subsequent instructions.

Example:

Before Optimization:

a$ = 10

b# = 20

t1 = a$ + b#

After Optimization:

a$ = 10

b# = 20

t1 = 30

Redundant Assignment Removal

Assignments that become unnecessary are removed.

Example:

Before Optimization:

a$ = 0

a$ = 10

After Optimization:

a$ = 10

The compiler eliminates redundant initialization statements whenever possible.

Chapter 12: Graphical User Interface Design

A graphical user interface was developed using Tkinter.

The objective of the GUI is to provide a visual representation of all compilation phases.

The GUI contains seven major tabs.

Source Code Tab

Allows users to enter source code.

Example:

int a$;

a$ = 10;

Tokens Tab

Displays generated tokens.

Example:

INT int

ID a$

SEMICOLON ;

Parse Trace Tab

Displays parser activities.

Example:

[INFO] Parsing Started

[DECLARATION] int a$

[ASSIGNMENT] a$

[SUCCESS] Parsing Completed

Symbol Table Tab

Displays variables and their values.

Example:

a$ -> 10

b# -> 20

Intermediate Code Tab

Displays generated Three Address Code.

Example:

t1 = a$ + b#

c@ = t1

Optimized Code Tab

Displays optimized instructions.

Example:

t1 = 30

c@ = 30

AST Tree Tab

Displays the hierarchical Abstract Syntax Tree.

Example:

ProgramNode

  AssignNode

    BinOpNode

      VarNode

Chapter 13: Testing and Validation

Extensive testing was performed to verify compiler correctness.

Several test cases were used.

Test Case 1

Input:

int a$;

a$ = 10;

Expected Result:

Successful compilation.

Output:

a$ = 10

Status:

PASS

Test Case 2

Input:

int a$;

int b#;

a$ = 10;

b# = 20;

Expected Result:

Successful compilation.

Status:

PASS

Test Case 3

Input:

int a$;

int b#;

int c@;

a$ = 10;

b# = 20;

c@ = a$ + b#;

Expected Result:

c@ = 30

Status:

PASS

Test Case 4

Input:

int a$;

int b#;

int c@;

a$ = 10;

b# = 20;

c@ = (a$ + b#) * 2;

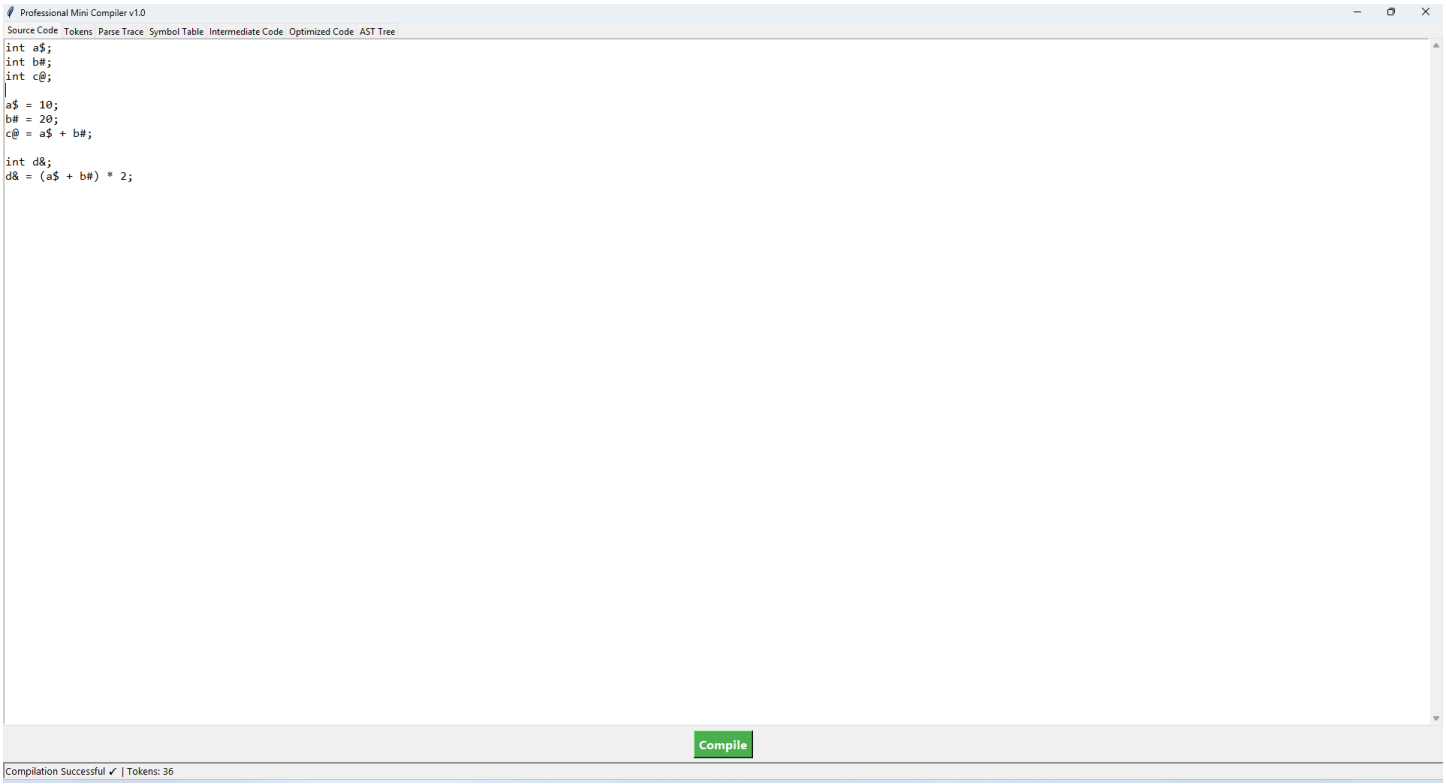
Expected Result:

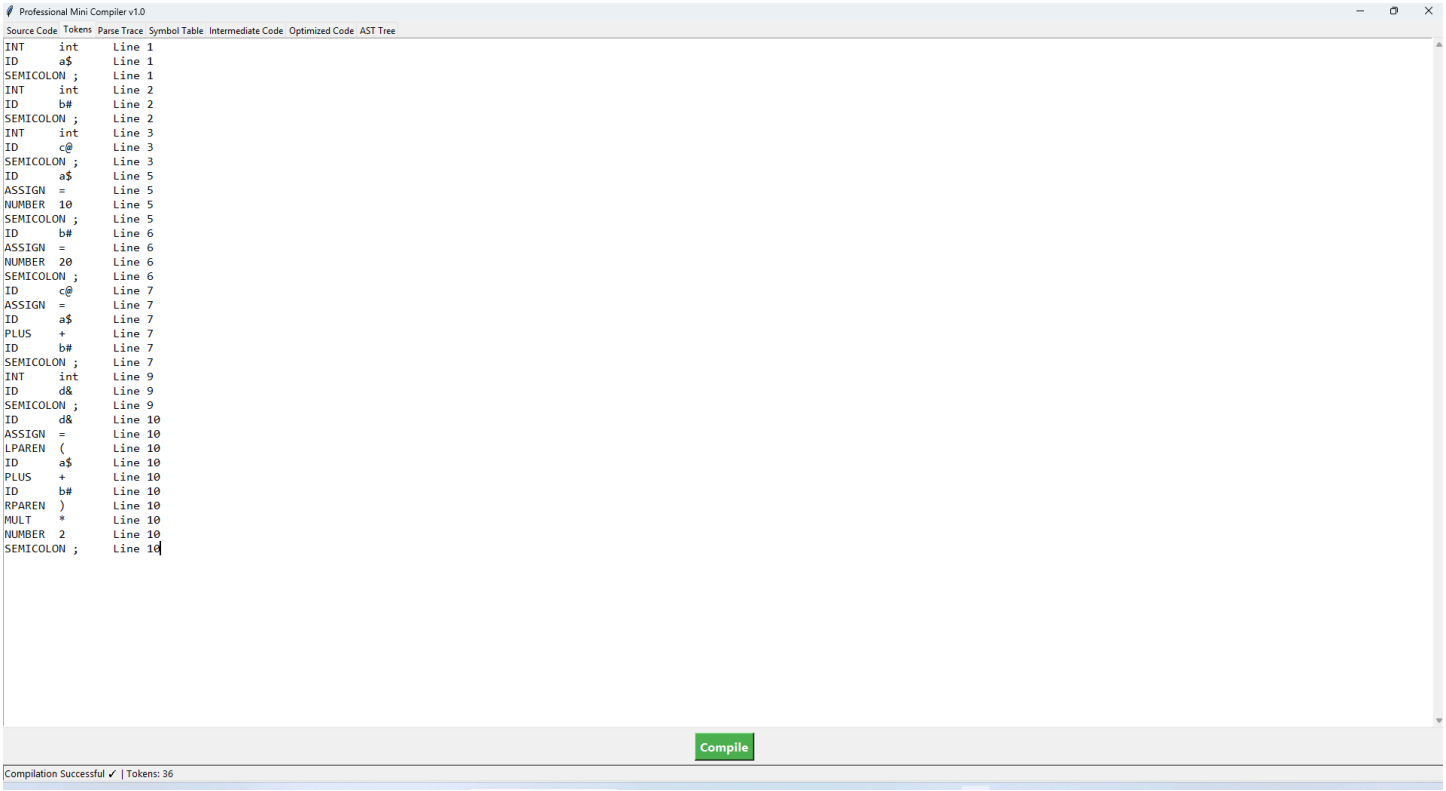
c@ = 60

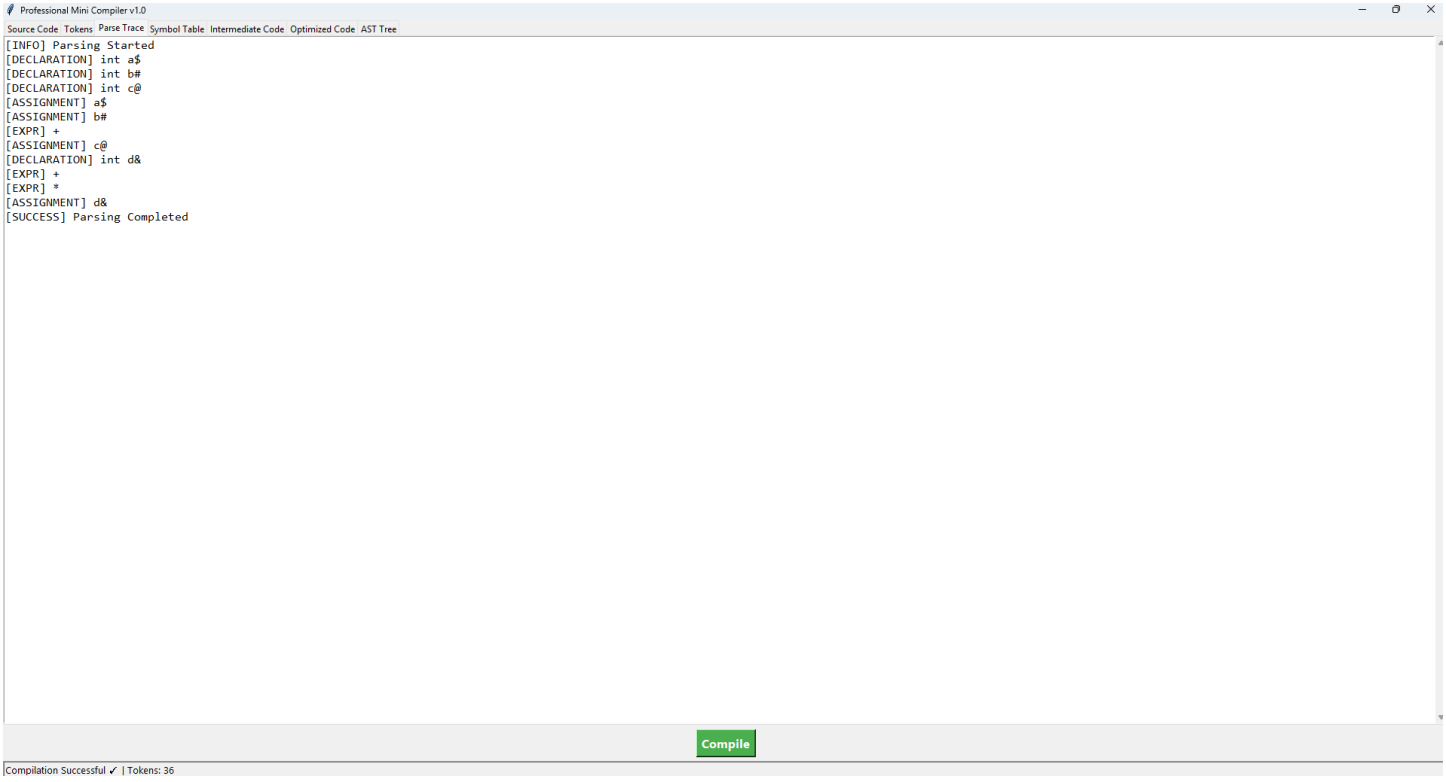
Status:

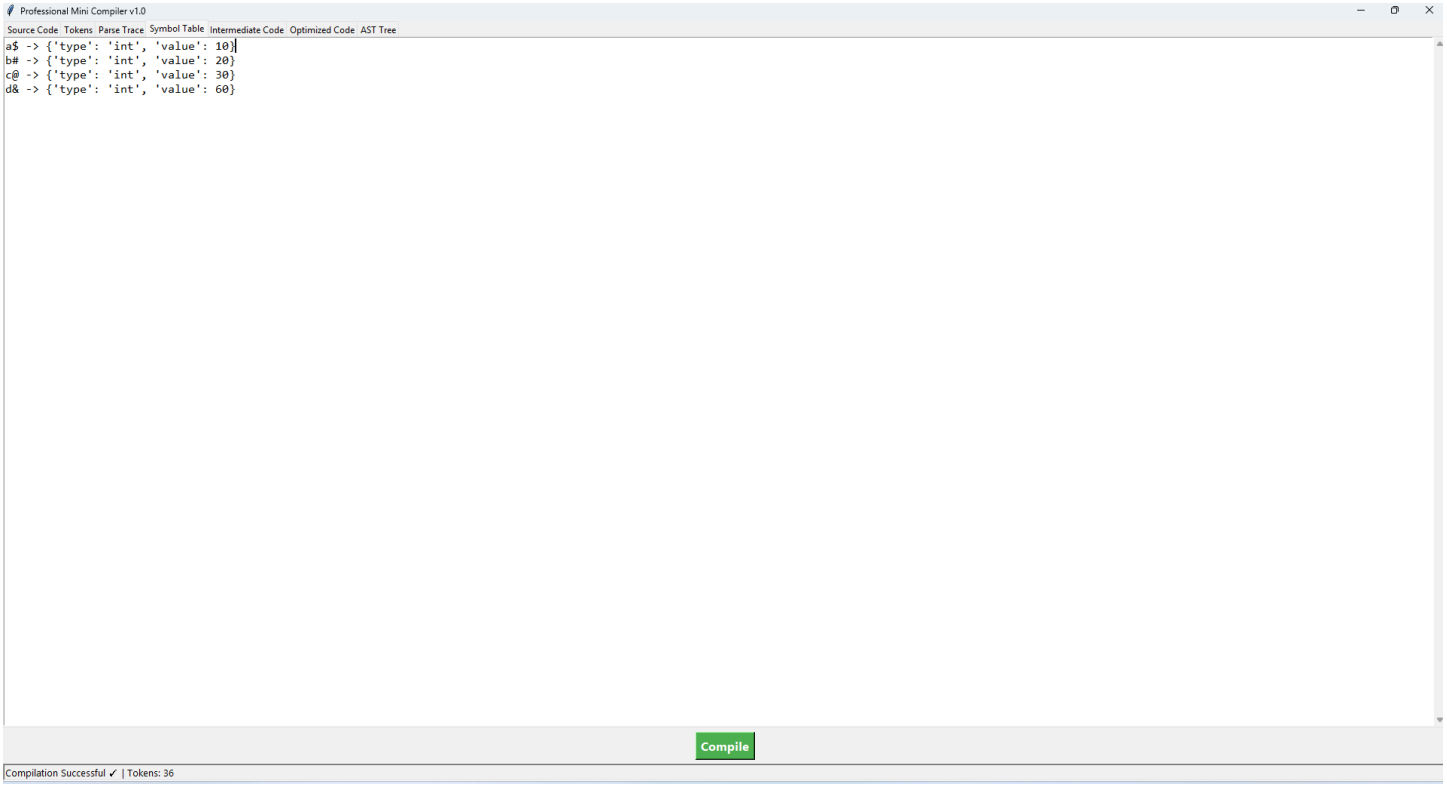
PASS

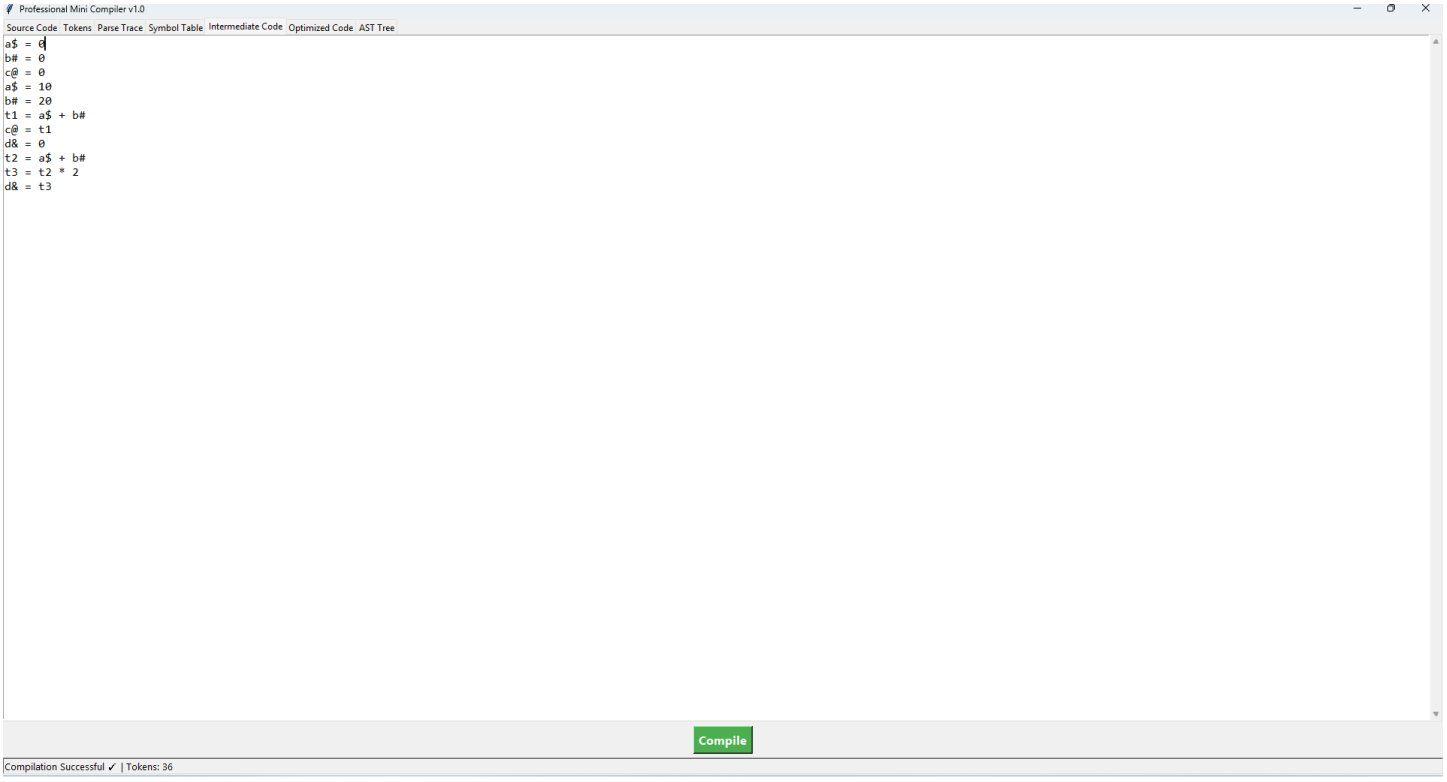
Chapter 14: Results and Discussion

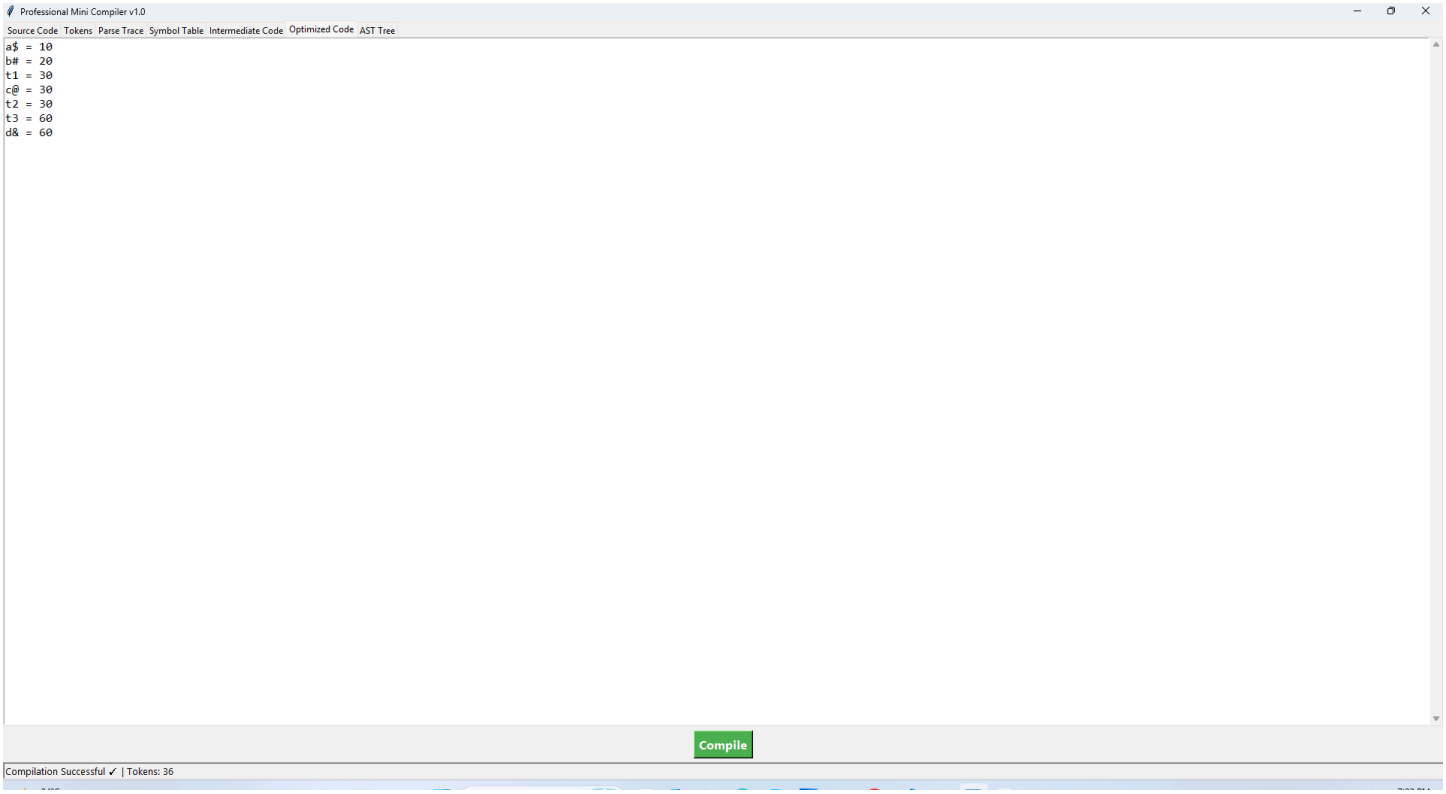


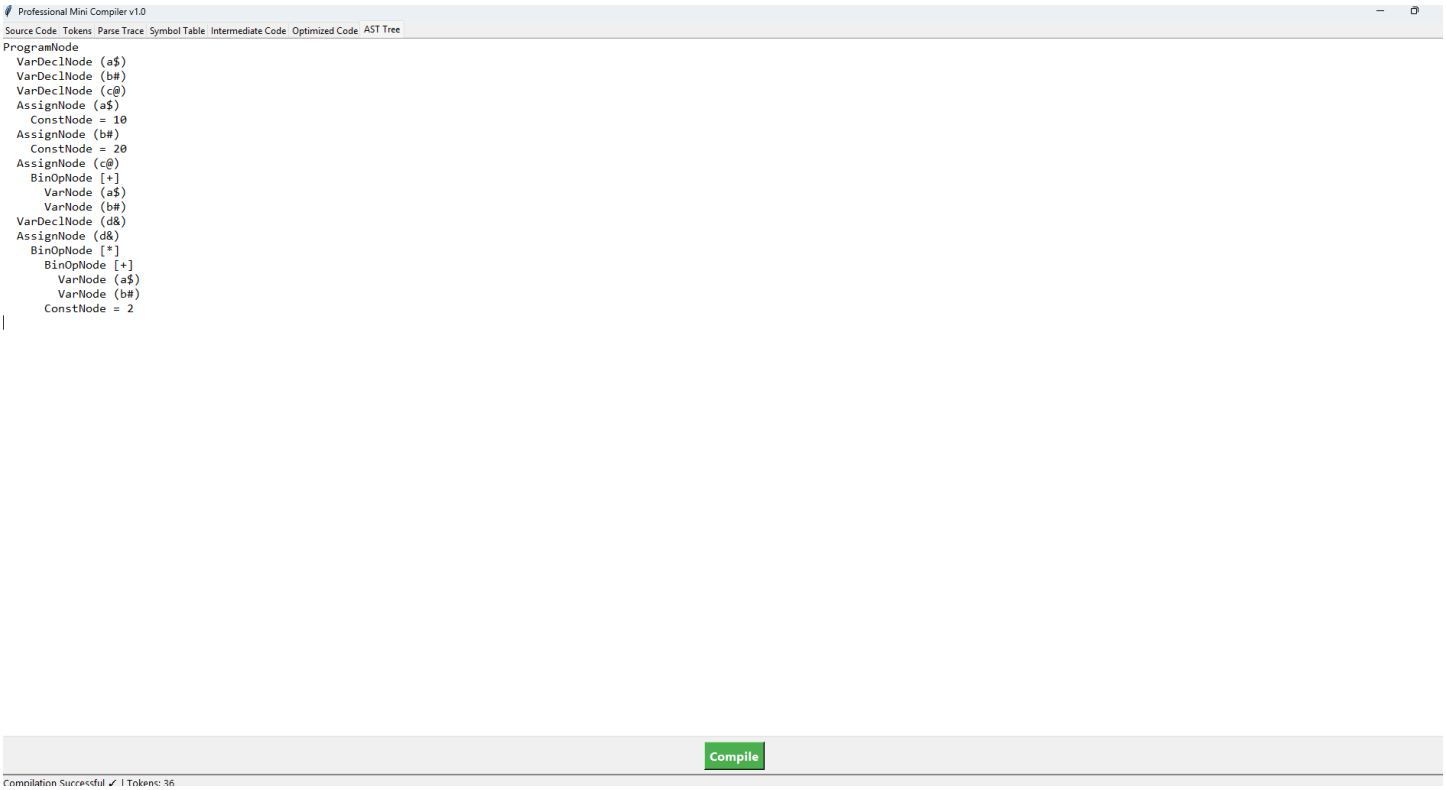

Professional Mini Compiler v1.0
Source Code
Tokens
Parse Trace
Symbol Table
Intermediate Code
Optimized Code
AST Tree
INT int Line 1
ID a$ Line 1
SEMICOLON ; Line 1
INT int Line 2
ID b# Line 2
SEMICOLON ; Line 2
INT int Line 3
ID c@ Line 3
SEMICOLON ; Line 3
ID a$ Line 5
ASSIGN = Line 5
NUMBER 10 Line 5
SEMICOLON ; Line 5
ID b# Line 6
ASSIGN = Line 6
NUMBER 20 Line 6
SEMICOLON ; Line 6
ID c@ Line 7
ASSIGN = Line 7
ID a$ Line 7
PLUS + Line 7
ID b# Line 7
SEMICOLON ; Line 7
INT int Line 9
ID d& Line 9
SEMICOLON ; Line 9
ID d& Line 10
ASSIGN = Line 10
LPAREN (Line 10
ID a$ Line 10
PLUS + Line 10
ID b# Line 10
RPAREN) Line 10
MULT * Line 10
NUMBER 2 Line 10
SEMICOLON ; Line 10
Compile
Compilation Successful ✓ | Tokens: 36


Professional Mini Compiler v1.0
Source Code
Tokens
Parse Trace
Symbol Table
Intermediate Code
Optimized Code
AST Tree
[INFO] Parsing Started
[DECLARATION] int a$
[DECLARATION] int b#
[DECLARATION] int c@
[ASSIGNMENT] a$
[ASSIGNMENT] b#
[EXPR] +
[ASSIGNMENT] c@
[DECLARATION] int d&
[EXPR] +
[EXPR] *
[ASSIGNMENT] d&
[SUCCESS] Parsing Completed
Compile
Compilation Successful ✓ | Tokens: 36


Professional Mini Compiler v1.0
Source Code
Tokens
Parse Trace
Symbol Table
Intermediate Code
Optimized Code
AST Tree
a$ -> {'type': 'int', 'value': 10}
b# -> {'type': 'int', 'value': 20}
c@ -> {'type': 'int', 'value': 30}
d& -> {'type': 'int', 'value': 60}
Compile
Compilation Successful ✓ | Tokens: 36


Professional Mini Compiler v1.0
Source Code
Tokens
Parse Trace
Symbol Table
Intermediate Code
Optimized Code
AST Tree
a$ = 0
b# = 0
c@ = 0
a$ = 10
b# = 20
t1 = a$ + b#
c@ = t1
d& = 0
t2 = a$ + b#
t3 = t2 * 2
d& = t3
Compile
Compilation Successful ✓ | Tokens: 36





The Mini Compiler successfully completed all major compilation phases.

The lexer correctly identified:

• Keywords

• Identifiers

• Numbers

• Operators

• Delimiters

The parser successfully generated AST structures.

The symbol table maintained correct variable information.

Intermediate code generation produced valid Three Address Code.

Optimization reduced redundant instructions and performed constant folding.

The GUI displayed all compilation stages clearly.

One of the major achievements of the project was successful support for special-character-based variable names.

Variables such as:

a$

b#

c@

d&

were processed successfully throughout all compilation phases.

Chapter 15: Advantages of the Project

The project provides several advantages:

• Simple and easy-to-understand compiler architecture

• Complete compiler pipeline

• GUI-based visualization

• Support for arithmetic expressions

• Support for special character identifiers

• Automatic AST generation

• Intermediate code generation

• Code optimization

• Educational value

• Extensible architecture

Chapter 16: Limitations

Despite its success, the compiler has some limitations.

Current limitations include:

• Supports only integer data type

• No floating-point support

• No arrays

• No functions

• No loops

• No conditional statements

• No semantic error checking

• No machine code generation

These limitations were intentionally maintained to keep the project focused on core compiler concepts.

Chapter 17: Future Improvements

Future versions of this compiler may include:

• If-Else Statements

• For Loops

• While Loops

• Functions and Procedures

• Semantic Analysis

• Type Checking

• Register Allocation

• Assembly Code Generation

• Machine Code Generation

• Advanced Optimization Techniques

• Error Recovery Mechanisms

• Compiler Visualization Enhancements

These improvements would make the compiler more similar to industrial compiler systems.

Chapter 18: Conclusion

This project successfully demonstrates the implementation of a Mini Compiler using Python and PLY.

The compiler performs Lexical Analysis, Syntax Analysis, AST Generation, Symbol Table Construction, Intermediate Code Generation, and Code Optimization.

A professional GUI was developed to visualize each compilation phase.

The project also introduced support for identifiers containing special characters such as $, #, @, &, and *, demonstrating the flexibility of compiler design.

Through this project, practical knowledge of compiler construction, parsing techniques, abstract syntax trees, intermediate representations, and optimization strategies was acquired.

The implementation provides a strong foundation for understanding modern compiler architecture and can be extended into a full-scale compiler in future research and development work.

References

1. Aho, Lam, Sethi, Ullman – Compilers: Principles, Techniques and Tools (Dragon Book)
2. Python Official Documentation
3. PLY (Python Lex-Yacc) Documentation
4. Tkinter GUI Documentation
5. Compiler Design Lecture Notes